

IMPLEMENTATION SUMMARY

M.O.A.C. v3.1 — New AGI Subsystems

June 13, 2026 · Defensive Publication · Public Distribution

Author: Melvin E. Bellinger Jr. — Connecticut, United States

LICENSE AND PATENT RIGHTS

© 2026 Melvin E. Bellinger Jr.

DOCUMENT LICENSE — Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). You may share and adapt this document for non-commercial purposes with attribution.
<https://creativecommons.org/licenses/by-nc/4.0/>

PATENT RIGHTS RESERVED — All patent rights to the inventions, methods, systems, and architectures described herein are reserved exclusively by the author, Melvin E. Bellinger Jr. This public disclosure establishes prior art dated June 13, 2026, preventing any third party from obtaining patent rights over the described methods. No license to any patent right is granted to any party by this document. Attribution required in any reference or derivative work: Melvin E. Bellinger Jr., "Mother of All Creation (M.O.A.C.) v3.1," June 13, 2026.

OVERVIEW

Four new subsystems were designed and integrated into the M.O.A.C. system on June 13, 2026. Each is implemented as a standalone ES module, imported into the main server process, and wired into the per-query hot path. All four execute in parallel on every standard query turn; background learning tasks are dispatched post-response via `setImmediate` to avoid contributing to response latency.

Hardware context: Consumer-grade workstation — GTX 1650 4 GB GPU, Windows 11. All new systems run on CPU in JavaScript with zero GPU dependency, zero additional model downloads, and zero third-party API calls beyond what the base system already uses.

SYSTEM 1: AGI Core Pipeline

| Property | Value | |---|---| | **Module** | `agi-core-pipeline.js` | | **Hot-path cost** | ≤ 5 ms per query (no LLM calls) | | **Persistent state** | `data/agi-pipeline-state.json` |

Executes five AGI-class operations synchronously before each LLM call.

Metacognition Subsystem

Maintains a live confidence model of the system's own knowledge. Two domain arrays

(`UNCERTAIN_DOMAINS` , `HIGH_CONFIDENCE_DOMAINS`) classify each query. Confidence score computation:

```
Base confidence:           0.75
High-confidence domain match: +0.15
Uncertain domain match:    -0.25
Range:                     [0.10, 1.00]
```

Queries scoring below 0.50 are classified as **knowledge boundary events**. The boundary topic is appended to a rolling 30-entry log. A `[AGI-SELF]` directive is injected into the prompt, instructing the model to express explicit uncertainty rather than confabulate.

Causal Reasoning Subsystem

Detects six causal query types via pattern matching on every turn. Detection result determines the reasoning directive injected into the prompt context:

- Counterfactual queries → *"Reason about the alternate timeline explicitly."*
- All other causal types → *"Trace cause-to-effect chains explicitly."*

Theory of Mind Subsystem

Maintains a live user belief model updated every turn:

- Emotion detection** — 5-state classifier: `frustrated` / `positive` / `anxious` / `confused` / `seeking-help`
- Expertise inference** — 3-level classifier: `expert` / `intermediate` / `beginner` (derived from vocabulary signals)
- Per-state communication directives** injected into prompt context:
 - Expert users: *"Skip basics; use precise terminology."*
 - Beginner users: *"Explain clearly; avoid jargon."*

Explainability Subsystem

Every routing decision is logged as a typed, timestamped record:

```
{
  "type": "query-analysis",
  "query": "[first 80 characters of query]",
  "reason": "at-knowledge-boundary; causal(counterfactual); user-emotion:frustrated",
  "confidence": 0.35,
  "route": "cloud-groq",
  "causal": ["counterfactual"],
  "emotion": "frustrated",
  "ts": 1749823461000
}
```

Last 25 records retained. Constitutes a machine-readable audit trail of every reasoning decision the system makes.

Self-Preservation Subsystem

30-second cached health monitor with two checks:

- **Disk viability** — write and delete a sentinel file in the `data/` directory
- **Memory pressure** — `process.memoryUsage().heapUsed` threshold at 3.5 GB (approaching the 4 GB Node.js heap limit)

Warnings are injected into the prompt context as `[AGI-HEALTH]` directives when a threat condition is detected.

SYSTEM 2: Persistent Episodic and Semantic Memory

Property Value --- ---	Module <code>persistent-episodic-memory.js</code>	Hot-path cost < 2 ms retrieval
(TF-IDF cosine similarity; no model)	Persistent state <code>data/episodic-memory.json</code> , <code>data/semantic-memory.json</code>	

Episodic Tier

Each conversation exchange is stored as an episode record:

Field Description --- ---	<code>episode_id</code> Unique identifier	<code>timestamp</code> Unix milliseconds
<code>session_id</code> Conversation session	<code>query</code> Query excerpt (≤ 300 characters)	<code>answer</code> Answer excerpt (≤ 600 characters)
<code>emotion</code> Emotional state at time of exchange	<code>topics</code> Up to 8 topic tokens	<code>tf</code> Term-frequency vector (used for retrieval)

Capacity: 2,000 episodes. **Retrieval:** Cosine similarity of the incoming query TF vector against all stored episode TF vectors. Minimum similarity threshold: 0.12. Returns top-N episodes sorted by score.

Semantic Tier

Facts extracted from responses via declarative sentence detection. Storage record:

| Field | Description | |---|---| | `fact` | Declarative sentence (≤ 300 characters) | | `topics` | Associated topic tokens | | `confidence` | Range [0, 1]; incremented on re-observation | | `tf` | TF vector for retrieval |

Deduplication: Facts with cosine similarity > 0.75 against an existing entry are merged (`confidence` += 0.05) rather than stored as duplicates. **Capacity:** 500 facts.

TF-IDF Implementation

No external library. Implementation details:

- Stop-word list: 63 high-frequency English words
- Tokenization: lowercased, punctuation stripped, tokens ≤ 2 characters dropped, stop-words removed
- Term frequency normalized by total token count
- Cosine similarity: standard dot-product divided by ($|A| \times |B|$)

Context Injection Format

```

🗄️ EPISODIC MEMORY (similar past exchanges):
• [4 min ago] Q: [query excerpt] → A: [answer excerpt]
• [12 min ago] Q: [query excerpt] → A: [answer excerpt]

💡 SEMANTIC MEMORY (learned facts):
• [fact text] (confidence: 84%)
• [fact text] (confidence: 70%)
  
```

SYSTEM 3: Continual Knowledge Graph

| Property | Value | |---|---| | **Module** | `continual-knowledge-graph.js` | | **Hot-path cost** | < 3 ms query (hash table lookups) | | **Persistent state** | `data/knowledge-graph.json` |

Graph Structure

Directed, weighted, typed property graph:

- **Nodes:** concept strings $\rightarrow$ `{ count, firstSeen, lastSeen, domains }`
- **Edges:** `conceptA  $\rightarrow$  conceptB  $\rightarrow$  { type, weight, firstSeen, lastSeen }`
- Edge weight increments each time the same relation is observed
- Node domain set expands as concept appears in new domains

- **Scale limits:** 3,000 nodes, 8,000 edges (LRU pruning by `node.count`)

Concept Extraction

Per turn: tokenize query + answer, extract single tokens (length ≥ 3 , non-stop-word) and adjacent non-stop-word bigrams. Up to 20 concepts per turn. All concepts co-occurring in the same turn receive bidirectional `co-occurs` edges with incrementing weight.

Typed Relation Extraction

Six typed relations extracted from answer text via regex:

```
is-a           "X is a/an Y"
causes         "X causes / leads to / results in Y"
has-part       "X contains / includes / has Y"
depends-on      "X requires / depends on / needs Y"
contradicts    "X contradicts / opposes / conflicts with Y"
similar-to     "X similar to / like / resembles Y"
```

Typed edges are added only when at least one endpoint is already a known node, preventing sparse noise from polluting the graph with one-off extractions.

Transfer Learning Mechanism

`findTransferConcepts(domain)` returns all concepts with `domains.size  $\geq 2$` AND `count  $\geq 3$` that include the specified domain. These are concepts encountered across at least two distinct knowledge domains — the operational definition of transferable knowledge in this implementation.

Domain Classification

Seven domains via regex: `programming`, `science`, `ai-ml`, `humanities`, `health`, `mathematics`, `business`. Default: `general`.

Context Injection Format

```

  KNOWLEDGE GRAPH (related concepts):
  neural[co-occurs, domain:ai-ml], training[causes, domain:ai-ml], ←depends-
  on:architecture[domain:ai-ml]
```

SYSTEM 4: Curiosity and Goal Engine

| Property | Value | |---|---| | **Module** | `curiosity-goal-engine.js` | | **Hot-path cost** | < 1 ms (state read + string construction) | | **Persistent state** | `data/curiosity-state.json` |

Novelty Score Model

Each tracked topic carries a curiosity score in [0.05, 1.0]:

```
Initial value:      1.0    (never discussed = maximally novel)
Per-visit decay:    × 0.95
Time restoration:   min(0.3, hours_since_last_visit × 0.0125)
                    (24 hours of absence → +0.3 restoration)
Floor:              0.05  (no topic ever becomes entirely stale)
```

This model captures the intuition that a topic becomes less novel with repeated discussion but regains interest after a period of absence — analogous to the spacing effect in human memory.

Knowledge Gap Detection

Post-response pattern scan against four hedge-language signal groups:

- **Group 1:** "I don't know", "I'm not sure", "I'm uncertain", "unclear to me", "beyond my", "can't confirm", "not certain"
- **Group 2:** "you might want to", "you could research", "worth looking up", "I suggest checking", "I recommend verifying"
- **Group 3:** "my knowledge may be outdated / limited / incomplete"
- **Group 4:** "I lack specific / detailed / current / real-time information"

One match triggers a gap record (deduplicated by topic prefix).

Autonomous Goal Generation

Eight goal templates filled with topic and domain variables. Two trigger paths:

1. **Gap-triggered** (priority 0.8) — fires immediately on knowledge gap detection
2. **Curiosity-triggered** (priority 0.5) — fires every 5 turns, seeded by highest-novelty tracked topic

Goal capacity: 20 active goals (oldest dropped when exceeded).

Example goal forms (illustrative):

- *"Explore the connections between [topic] and [domain] across domains"*
- *"Challenge my assumptions about [topic] — what might I be wrong about?"*
- *"Identify the open questions in [topic] that no one has fully answered"*

Curiosity Pulse

`getCuriosityPulse()` merges the top 3 high-novelty tracked topics with the top 2 recent knowledge gap topics into a deduplicated list of up to 3 topics. This represents what the system is currently most "curious about" in an operationally meaningful sense.

Context Injection Format

 CURIOSITY PULSE (currently most curious about): `topic1, topic2, topic3`

 ACTIVE EXPLORATION GOALS:

- Explore the connections between `[topic]` and `[domain]` across domains
- Challenge my assumptions about `[topic]` – what might I be wrong about?

INTEGRATION SUMMARY

Hot-Path Execution Order (per standard query turn)

| Step | Operation | Module | Output | |---|---|---|---| | 1 | `runAGIPipeline()` | `agi-core-pipeline.js` | Metacognition + causal + ToM + health context block | | 2 | `buildEpisodicContext()` | `persistent-episodic-memory.js` | Past episodes + semantic facts block | | 3 | `buildGraphContext()` | `continual-knowledge-graph.js` | Related concepts block | | 4 | `buildCuriosityContext()` | `curiosity-goal-engine.js` | Curiosity pulse + goals block |

Background Execution (post-response, via `setImmediate`)

| Operation | Module | Effect | |---|---|---| | `storeEpisode()` | `persistent-episodic-memory.js` | Writes new episode to episodic store | | `ingestKnowledgeGraph()` | `continual-knowledge-graph.js` | Adds concepts and relations to graph | | `processCuriosityTurn()` | `curiosity-goal-engine.js` | Updates novelty scores; detects gaps; generates goals | | `extractEpisodicFacts()` | `persistent-episodic-memory.js` | Extracts and stores semantic facts from response |

All post-response operations execute after the HTTP response has been sent and add zero latency to the user-facing response.

OBSERVABLE STATE

Endpoint: `GET http://localhost:3002/api/v3/agi/state`

Returns a unified live JSON snapshot of all four subsystem states:

```
{
  "success": true,
  "agi": {
    "episodicMemory": {
      "episodeCount": 0,
      "semanticFactCount": 0,
      "oldestEpisodeAgo": 0,
      "topTopics": ["topic(count)", "..."]
    },
    "knowledgeGraph": {
      "nodeCount": 0,
      "edgeCount": 0,
      "turnCount": 0,
      "topConcepts": ["concept(count)", "..."]
    },
    "curiosity": {
      "totalExchanges": 0,
      "trackedTopics": 0,
      "activeGoals": [],
      "knowledgeGaps": [],
      "curiosityPulse": ["topic1", "topic2", "topic3"],
      "dominantDomain": "programming"
    },
    "systemHealth": {
      "ok": true,
      "warnings": []
    }
  }
}
```

This endpoint provides a real-time, machine-readable window into the system's internal cognitive state — a property not exhibited by standard LLM wrappers or chat applications.

Defensive publication. Intended for public distribution. — Melvin E. Bellinger Jr., June 13, 2026